

Group (9)

Ahmed Tarek , Omer Amer ,Yasir Aljaf , Mohamed AlAwadi

Mini-Project: Multithreaded Producer-Consumer Application

Project Overview:

In this project, you will develop a **multithreaded Producer-Consumer application** that utilizes a **bounded buffer** to manage data exchange between threads. The primary goal is to implement synchronization mechanisms using **semaphores** and **mutexes** to ensure data integrity and prevent race conditions.

Learning Objectives:

1. Understand the concepts of **multithreading** and **synchronization** in operating systems.
2. Apply **semaphores** and **mutexes** to solve concurrency problems.
3. Manage **bounded buffers** and implement inter-thread communication.

Project Requirements:

1. **Bounded Buffer Implementation:**
 - Create a shared buffer with a fixed size (e.g., 10 slots).
 - Use semaphores to manage the number of available slots and filled slots.
 - Use a mutex to ensure mutual exclusion when accessing the buffer.
2. **Producer Threads:**
 - Each producer thread will generate random data (e.g., integers) and add them to the buffer.
 - The number of producers should be configurable (e.g., through command-line arguments).
 - Implement a mechanism to pause if the buffer is full.
3. **Consumer Threads:**
 - Each consumer thread will remove data from the buffer and process it (e.g., print it to the console or write it to a file).
 - The number of consumers should be configurable.
 - Implement a mechanism to pause if the buffer is empty.
4. **Synchronization and Safety:**
 - Use semaphores to track buffer capacity and prevent overflows/underflows.
 - Ensure no **race conditions** occur when multiple threads access the buffer.
 - Avoid **deadlocks** and ensure efficient resource utilization.
5. **Graceful Termination:**
 - Implement a mechanism for cleanly terminating producer and consumer threads after a specific condition is met (e.g., after producing/consuming a fixed number of items).
 - Ensure all resources (mutexes, semaphores) are appropriately released.
6. **Input Validation and Error Handling:**

- Validate command-line inputs (e.g., number of producers/consumers, buffer size).
- Handle errors gracefully (e.g., thread creation failure).

Bonus Features (Optional):

- Implement a **priority-based consumer** where certain data items are processed first.
- Add **logging** to track thread activities, buffer state, and synchronization events.

Deliverables:

1. **Source Code:** Well-documented and organized code with explicit comments explaining synchronization logic.
2. **README File:** Instructions on how to compile, run, and test the application. Include examples of input parameters and expected outputs.
3. **Report:** A brief report (1-2 pages) covering:
 - Design decisions (e.g., why specific synchronization mechanisms were chosen).
 - Challenges faced and how they were addressed.
4. **Demonstration Video:** A short video (2–5 minutes) showcasing the application in action and explaining key features. The video must include at least **two group members** presenting or narrating. You should walk through how the application works, highlight your synchronization mechanisms, and briefly explain any major challenges you overcame. The video can be recorded using screen recording tools (e.g., Zoom) or even a phone.

Grading Criteria:

1. **Correctness (50%):** The application meets the functional requirements and handles synchronization correctly.
2. **Code Quality (20%):** Code is clean, modular, and well-commented.
3. **Robustness (15%):** Proper error handling and input validation.
4. **Documentation (15%):** A clear README and report that explains the project.
5. **Bonus Features (10%):** Implementation of additional features beyond the basic requirements.

Submission Deadline:

- Submit all deliverables by **April 13** via **iLearn**.

Group Work Guidelines:

- Each group will consist of **4 members**. You can self-join your group on iLearn.
- Document individual contributions clearly in the report.
- Ensure equal participation and collaboration.

Additional Notes:

- **Language Requirement:** Students must implement the project in C using the POSIX pthread library. Make sure to include the appropriate headers (e.g., pthread.h, semaphore.h).
- **Termination Condition:** Each producer thread must generate a fixed number of items (e.g., 20), so the total number of items equals the number of producers $\times$ 20. Once all items are produced and consumed, all threads must terminate gracefully.
- **Test Case Suggestion:** Include in your README a test case such as:
./producer_consumer 3 2 10

This runs the program with 3 producer threads, 2 consumer threads, and a buffer size of 10. Clearly specify the number of items each producer generates and provide an example of the expected consumer output.

- **Compilation and Sample Output:**

Compile your code using:

```
gcc -o producer_consumer producer_consumer.c -pthread
```

Example usage:

```
./producer_consumer 3 2 10
```

This runs the program with **3 producer threads**, **2 consumer threads**, and a **buffer size of 10**.

Sample output:

```
[Producer-1] Produced item: 42  
[Producer-2] Produced item: 17  
[Consumer-1] Consumed item: 42  
[Producer-3] Produced item: 88  
[Consumer-2] Consumed item: 17  
...  
[Producer-1] Finished producing 20 items.  
[Consumer-1] Finished consuming.
```

You may also choose to display buffer indices if they are helpful. Ensure that your output is **clear and consistent**.

- **Group Formation Deadline:** All groups must be formed on iLearn by **March 27**. After this date, group changes will not be allowed.

Good luck!

Our Code:

```

/*
 * Multithreaded Producer-Consumer using POSIX pthreads, semaphores, and mutex
 * Group (9)
 * Ahmed Tarek , Omer Amer , Yasir al jaf , Mohamed Al Awadi
 * CMP-310 Operating Systems Mini Project
 */

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS_PER_PRODUCER 20

int *buffer;
int buffer_size;
int in = 0, out = 0;

sem_t empty_slots;
sem_t filled_slots;
pthread_mutex_t buffer_mutex;

int total_produced = 0;
int total_consumed = 0;
int total_items;

void *producer(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < ITEMS_PER_PRODUCER; ++i) {
        int item = rand() % 100;

        sem_wait(&empty_slots);
        pthread_mutex_lock(&buffer_mutex);

        buffer[in] = item;
        printf("[Producer-%d] Produced item: %d\n", id, item);
        in = (in + 1) % buffer_size;
        total_produced++;
    }
}

```

```

pthread_mutex_unlock(&buffer_mutex);
sem_post(&filled_slots);

usleep(100000); // sleep 100ms to simulate time
}
printf("[Producer-%d] Finished producing %d items.\n", id, ITEMS_PER_PRODUCER);
free(arg);
return NULL;
}

void *consumer(void *arg) {
    int id = *(int *)arg;
    while (1) {
        sem_wait(&filled_slots);
        pthread_mutex_lock(&buffer_mutex);

        if (total_consumed >= total_items) {
            pthread_mutex_unlock(&buffer_mutex);
            sem_post(&filled_slots);
            break;
        }

        int item = buffer[out];
        printf("[Consumer-%d] Consumed item: %d\n", id, item);
        out = (out + 1) % buffer_size;
        total_consumed++;

        pthread_mutex_unlock(&buffer_mutex);
        sem_post(&empty_slots);

        usleep(150000); // sleep 150ms
    }
    printf("[Consumer-%d] Finished consuming.\n", id);
    free(arg);
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        fprintf(stderr, "Usage: %s <num_producers> <num_consumers> <buffer_size>\n", argv[0]);
    }
}

```

```

    exit(1);
}

int num_producers = atoi(argv[1]);
int num_consumers = atoi(argv[2]);
buffer_size = atoi(argv[3]);
total_items = num_producers * ITEMS_PER_PRODUCER;

if (num_producers <= 0 || num_consumers <= 0 || buffer_size <= 0) {
    fprintf(stderr, "Invalid input: All values must be positive integers.\n");
    exit(1);
}

buffer = malloc(sizeof(int) * buffer_size);
pthread_t producers[num_producers];
pthread_t consumers[num_consumers];

sem_init(&empty_slots, 0, buffer_size);
sem_init(&filled_slots, 0, 0);
pthread_mutex_init(&buffer_mutex, NULL);

srand(time(NULL));

for (int i = 0; i < num_producers; ++i) {
    int *id = malloc(sizeof(int));
    *id = i + 1;
    pthread_create(&producers[i], NULL, producer, id);
}

for (int i = 0; i < num_consumers; ++i) {
    int *id = malloc(sizeof(int));
    *id = i + 1;
    pthread_create(&consumers[i], NULL, consumer, id);
}

for (int i = 0; i < num_producers; ++i) {
    pthread_join(producers[i], NULL);
}

// Wake up consumers so they can exit
while (1) {

```

```
pthread_mutex_lock(&buffer_mutex);
if (total_consumed >= total_items) {
    pthread_mutex_unlock(&buffer_mutex);
    for (int i = 0; i < num_consumers; ++i) sem_post(&filled_slots);
    break;
}
pthread_mutex_unlock(&buffer_mutex);
}

for (int i = 0; i < num_consumers; ++i) {
    pthread_join(consumers[i], NULL);
}

sem_destroy(&empty_slots);
sem_destroy(&filled_slots);
pthread_mutex_destroy(&buffer_mutex);
free(buffer);

return 0;
}
```

Note : I used Kali Linux as my virtual machine as I am used to it

Testing Our Code:

```
(kali㉿kali)-[~]  
└─$ nano producer_consumer.c
```

```
(kali㉿kali)-[~]  
└─$ gcc -o producer_consumer producer_consumer.c -pthread
```

No Errors , Means Compiled Successfully

```
(kali㉿kali)-[~]  
└─$ ./producer_consumer 3 2 10
```

OUTPUT:

```
[Producer-2] Produced item: 67  
[Producer-3] Produced item: 12  
[Producer-1] Produced item: 77  
[Consumer-1] Consumed item: 67  
[Consumer-2] Consumed item: 12  
[Producer-3] Produced item: 45  
[Producer-2] Produced item: 69  
[Producer-1] Produced item: 11  
[Consumer-2] Consumed item: 77  
[Consumer-1] Consumed item: 45  
[Producer-2] Produced item: 14  
[Producer-3] Produced item: 40  
[Producer-1] Produced item: 67  
[Consumer-2] Consumed item: 69  
[Consumer-1] Consumed item: 11  
[Producer-3] Produced item: 64  
[Producer-2] Produced item: 55  
[Producer-1] Produced item: 93  
[Producer-2] Produced item: 27  
[Producer-3] Produced item: 20
```

[Producer-1] Produced item: 9
[Consumer-1] Consumed item: 14
[Consumer-2] Consumed item: 40
[Producer-2] Produced item: 2
[Producer-3] Produced item: 71
[Producer-1] Produced item: 74
[Consumer-2] Consumed item: 67
[Consumer-1] Consumed item: 64
[Producer-3] Produced item: 65
[Producer-1] Produced item: 87
[Consumer-2] Consumed item: 55
[Producer-2] Produced item: 35
[Consumer-1] Consumed item: 93
[Producer-3] Produced item: 4
[Consumer-2] Consumed item: 27
[Consumer-1] Consumed item: 20
[Producer-3] Produced item: 13
[Producer-1] Produced item: 90
[Consumer-2] Consumed item: 9
[Producer-2] Produced item: 53
[Consumer-1] Consumed item: 2
[Producer-1] Produced item: 93
[Consumer-2] Consumed item: 71
[Producer-3] Produced item: 11
[Consumer-1] Consumed item: 74
[Producer-2] Produced item: 41
[Consumer-2] Consumed item: 65
[Producer-1] Produced item: 16
[Consumer-1] Consumed item: 87
[Producer-2] Produced item: 75
[Consumer-2] Consumed item: 35
[Producer-3] Produced item: 77
[Consumer-1] Consumed item: 4
[Producer-1] Produced item: 35
[Consumer-2] Consumed item: 13
[Producer-2] Produced item: 87
[Consumer-1] Consumed item: 90
[Producer-3] Produced item: 54
[Consumer-2] Consumed item: 53
[Producer-1] Produced item: 80
[Consumer-1] Consumed item: 93

[Producer-3] Produced item: 8
[Consumer-2] Consumed item: 11
[Producer-2] Produced item: 17
[Consumer-1] Consumed item: 41
[Producer-1] Produced item: 47
[Consumer-2] Consumed item: 16
[Producer-3] Produced item: 1
[Consumer-1] Consumed item: 75
[Producer-2] Produced item: 85
[Consumer-2] Consumed item: 77
[Producer-1] Produced item: 11
[Consumer-2] Consumed item: 35
[Producer-3] Produced item: 56
[Consumer-1] Consumed item: 87
[Producer-2] Produced item: 78
[Consumer-2] Consumed item: 54
[Producer-1] Produced item: 39
[Consumer-1] Consumed item: 80
[Producer-2] Produced item: 39
[Consumer-2] Consumed item: 8
[Producer-3] Produced item: 28
[Consumer-1] Consumed item: 17
[Producer-1] Produced item: 93
[Consumer-2] Consumed item: 47
[Producer-2] Produced item: 51
[Consumer-2] Consumed item: 1
[Producer-3] Produced item: 13
[Consumer-2] Consumed item: 85
[Producer-1] Produced item: 58
[Consumer-1] Consumed item: 11
[Producer-3] Produced item: 38
[Consumer-2] Consumed item: 56
[Producer-2] Produced item: 0
[Consumer-1] Consumed item: 78
[Producer-1] Produced item: 14
[Consumer-2] Consumed item: 39
[Producer-3] Produced item: 81
[Consumer-1] Consumed item: 39
[Producer-2] Produced item: 80
[Consumer-2] Consumed item: 28
[Producer-1] Produced item: 53

[Consumer-1] Consumed item: 93
[Producer-3] Produced item: 74
[Consumer-2] Consumed item: 51
[Producer-2] Produced item: 64
[Producer-3] Finished producing 20 items.
[Consumer-1] Consumed item: 13
[Producer-2] Produced item: 42
[Consumer-2] Consumed item: 58
[Producer-1] Produced item: 21
[Producer-2] Finished producing 20 items.
[Producer-1] Finished producing 20 items.
[Consumer-1] Consumed item: 38
[Consumer-2] Consumed item: 0
[Consumer-1] Consumed item: 14
[Consumer-2] Consumed item: 81
[Consumer-1] Consumed item: 80
[Consumer-2] Consumed item: 53
[Consumer-1] Consumed item: 74
[Consumer-2] Consumed item: 64
[Consumer-1] Consumed item: 42
[Consumer-2] Consumed item: 21
[Consumer-1] Finished consuming.
[Consumer-2] Finished consuming.